

Rapport d'Architecture Cloud et Infrastructure

Projet Big Data - BigMedia

M2 Business Intelligence

1 INTRODUCTION

Ce document présente une analyse détaillée de l'architecture, de l'infrastructure et des ressources cloud mises en œuvre pour le projet BigMedia. L'objectif est de fournir une vue d'ensemble technique de la solution, en mettant l'accent sur l'utilisation des services Google Cloud Platform (GCP), l'Infrastructure as Code (IaC) et les pipelines de données modernes.

2 ARCHITECTURE GLOBALE

L'architecture du projet repose sur une approche « Cloud-Native », tirant parti des services managés de GCP pour assurer la scalabilité, la fiabilité et la maintenabilité.

Les composants clés sont :

- **Cloud Provider** : Google Cloud Platform (GCP).
- **Infrastructure as Code** : Terraform (OpenTofu) pour le provisionnement des ressources.
- **Orchestration** : Prefect pour la gestion des flux de données.
- **Transformation** : dbt (Data Build Tool) pour la modélisation et la transformation des données dans BigQuery.
- **Langage** : Python (gestion des dépendances avec uv).

3 INFRASTRUCTURE CLOUD (GCP)

L'infrastructure est entièrement définie en code (IaC) via Terraform, garantissant la reproductibilité et la traçabilité des environnements. Les fichiers de configuration se trouvent dans le répertoire `infrastructure/`.

3.1 Ressources Provisionnées

Les ressources principales déployées sur GCP sont les suivantes :

1. Stockage (Google Cloud Storage) :

- `terraform-state` : Bucket pour stocker l'état de Terraform (backend distant).
- `ingested-data-dev` : Bucket pour les données brutes en environnement de développement.
- `ingested-data-prod` : Bucket pour les données brutes en environnement de production.

2. Data Warehouse (BigQuery) :

- `bi_dataset_dev` : Dataset pour l'environnement de développement (expiration des tables configurée à 90 jours).
- `bi_dataset_prod` : Dataset pour l'environnement de production.

3. Gestion des Identités et Accès (IAM) :

- Création d'un compte de service dédié pour dbt (`projet-m2-bi-dbt-sa`).
- Attribution des rôles nécessaires selon le principe du moindre privilège :
 - `roles/bigquery.dataEditor` : Pour créer et modifier des tables.
 - `roles/bigquery.jobUser` : Pour exécuter des requêtes.
 - `roles/storage.admin` : Pour gérer les objets dans les buckets GCS.

3.2 Configuration Terraform

Le fichier `main.tf` définit ces ressources en utilisant le provider `hashicorp/google`. La configuration supporte plusieurs environnements via des variables (`variables.tf`), permettant de séparer proprement les ressources de développement et de production.

4 PIPELINE DE DONNÉES

Le pipeline de données est conçu pour transformer les données brutes en informations exploitables pour la BI.

4.1 Orchestration avec Prefect

Prefect est utilisé pour orchestrer les tâches. Le fichier `prefect_flows/pipeline.py` définit le flow principal `run_dbt_models`.

- **Flexibilité** : Le pipeline supporte deux modes d'exécution :
 - **Cloud** : Utilise les « Blocks » Prefect pour stocker les configurations et profils dbt.
 - **Local** : Utilise un fichier `profiles.yml` généré localement pour le développement.
- **Robustesse** : Les tâches sont configurées avec des mécanismes de « retry » pour gérer les erreurs transitoires.

4.2 Transformation avec dbt

dbt est le cœur de la transformation des données. Le projet est structuré en couches logiques (Architecture Medallion) :

1. **Staging (staging)** : Vues matérialisant les données brutes, nettoyage léger et renommage des colonnes.
2. **Base (base)** : Couche intermédiaire pour des transformations communes.
3. **Intermediate (intermediate)** : Logique métier complexe, jointures.
4. **Marts (marts)** : Tables finales optimisées pour l'analyse (schéma en étoile), prêtes à être consommées par les outils de BI.

Le projet dbt (`dbt_project.yml`) configure ces couches avec des matérialisations spécifiques (vues pour staging/base, tables pour marts).

5 MÉTHODOLOGIE ET OUTILS

5.1 Gestion de l'Environnement

Le projet utilise `uv` pour une gestion rapide et fiable des dépendances Python. Le fichier `pyproject.toml` liste toutes les bibliothèques nécessaires (`dbt-bigquery`, `prefect-gcp`, etc.), assurant que tous les développeurs travaillent avec les mêmes versions d'outils.

5.2 Séparation des Environnements

Une distinction stricte est maintenue entre les environnements de développement (`dev`) et de production (`prod`) :

- Buckets GCS distincts.
- Datasets BigQuery distincts.
- Profils dbt configurables via la ligne de commande.

Cette séparation permet de développer et tester les nouvelles fonctionnalités sans impacter les données de production.

6 CONCLUSION

L'architecture mise en place pour le projet BigMedia démontre une utilisation mature des technologies cloud modernes. L'utilisation combinée de Terraform pour l'infrastructure, de Prefect pour l'orchestration et de dbt pour la transformation offre une solution robuste, scalable et facile à maintenir. L'approche « Code-First » (IaC, DataOps) garantit la qualité et la fiabilité des données livrées.